quality feedback loop. In enterprise settings, organizational mandates, sunk training costs, and managerial oversight might sustain AI tool use despite user frustration, potentially leading to distinct temporal patterns (as indicated in a recent study [73]). Similarly, enterprise quality assurance processes—mandatory code review, automated testing requirements, dedicated QA teams, and even dedicated agents to do maintenance work—might prevent proportional technical debt accumulation by catching issues before they escalate. Future research should examine whether the transient gains and proportional debt patterns we observe generalize to enterprise contexts or represent open-source-specific phenomena.

5.2 Practical Implications

To overcome the technical debt accumulation ratchet, software projects using LLM agent assistants should focus on process adaptation that scales quality assurance with AI-era velocity. The proportional technical debt accumulation we observe (Section 4.1.2), combined with its velocity-dampening effects (Section 4.2), creates a self-reinforcing cycle that needs to be addressed at the project level. To overcome this, AI-adopting teams may consider refactoring sprints triggered by code quality metrics, mandating test coverage requirements that scale with lines of code added, or prompt engineering (e.g., engineered Cursor rules) to enforce rigid quality standards for LLM agents. Without such adaptations, the initial productivity surge may accelerate the project toward an unmaintainable end state.

To support the above process adaptation, AI coding tools need explicit design to support quality assurance alongside code generation. The LLM agents (at the time of study) are generation-first, leaving quality maintenance as an afterthought. Future assistants should suggest tests alongside code, flag unnecessary complexity in real time, and proactively recommend refactoring when code quality degrades—essentially becoming "pair programmers" for quality, not just velocity. More provocatively, tools might implement self-throttling: automatically reducing suggestion volume or aggressiveness when project-level complexity or debt exceeds healthy thresholds, forcing developers to consolidate before generating more code. Such features would align tools with long-term project health rather than short-term code production.

The potential overcomplication in AI-generated code warrants further research and improvement. The 25% increase in code complexity we observe (Table 3) represents a distinct quality dimension beyond code quality issues—a "comprehension tax" that persists regardless of functional correctness. This suggests LLMs may be generating structurally valid but semantically opaque code, perhaps because training objectives prioritize passing tests over non-functional requirements such as human readability [57, 112]. Unless future development workflows allow fully automated AI development without any human code reviews, code readability will remain an important dimension to pursue in AI-generated code. Addressing this requires both technical innovation (e.g., readability-aware fine-tuning, post-hoc simplification passes) and empirical investigation into what specifically makes LLMs generate overly complicated implementations. Until these complexities are addressed, software project teams should treat AI-generated code as requiring extra scrutiny during review, with particular attention to whether simpler implementations exist that achieve the same functionality.

6 Conclusion

This study presents the first large-scale empirical investigation of how LLM agent assistants impact real-world software development projects. Through a rigorous difference-in-differences design comparing Cursor-adopting repositories with matched control group repositories, complemented by dynamic panel GMM analysis, we provide evidence that challenges both unbridled optimism and categorical pessimism surrounding AI-assisted coding: Cursor adoption produces substantial but transient velocity gains alongside persistent increases in technical debt; such technical debt accumulation subsequently dampens future development velocity. Ultimately, our results suggest a self-reinforcing cycle where initial productivity surges give way to maintenance burdens.

However, several considerations suggest this picture may not be as bleak as it initially appears. First, our study captures a snapshot of rapidly evolving technology from mid-2024 to mid-2025, when LLM capabilities, agent designs, and developer practices are improving at unprecedented rates—future tools might be able to address the quality concerns we observed. Second, our quality metrics, while well-established in software engineering research, may not fully capture the multi-dimensional nature of code quality in AI-driven development. For example, complexity metrics were designed for human-written code; whether they appropriately penalize AI-generated patterns that are mechanically verifiable yet syntactically complex remains an open question. Third, the open-source context of our study may amplify both abandonment dynamics and quality concerns relative to enterprise settings with mandatory, dedicated quality assurance processes.

Looking forward, our findings point to clear research and practice directions (Section 5). Ultimately, this study demonstrates that realizing the promise of AI-assisted software development requires a holistic understanding of how AI assistance reshapes the fundamental trade-offs between development velocity, code quality, and long-term project sustainability. The age of AI coding has arrived—our challenge now is to harness it wisely.

Data Availability

We provide a replication package for this paper at:

https://doi.org/10.5281/zenodo.18368661

The Appendix is available in the latest arXiv version of this paper at: https://arxiv.org/abs/2511.04427.

Acknowledgments

He's, Kästner's, and Miller's work was supported in part by the National Science Foundation (award 2206859). Miller's work was also supported by the National Science Foundation Graduate Research Fellowship Program under Grant Number DGE214073. Vasilescu's and Agarwal's work was supported in part by the National Science Foundation (awards 2317168 and 2120323) and research awards from Google and the Digital Infrastructure Fund. We would like to thank Narayan Ramasubbu and Alexandros Kapravelos for providing valuable methodological feedback at earlier stages of this research. We would also like to thank all S3C2 Quarterly Meeting attendees for their insightful discussions around the early results of this study. Finally, we would like to thank Google Cloud for offering research credits to cover BigQuery-based analysis in this research.